



Assessment 2 : Programming Project

Lecturer: [Suirui ZHU](#)

Type: Individual

Unit Code: MIT102

Unit Name: MIT102 Operating Systems and Computer Architecture

Student Name: Anish Gurung

Student ID: aitic20251080

Declaration: I have read Academic Misconduct Policy and understand the consequences of committing academic misconduct as outlined in the policy. I declare this assessment is written in my own words.

Signature/s _



1. Introduction	2
2. System Design & Design Decisions	2
3. Memory States.....	3
4. Physical Memory Layout.....	4
5. Data Structures	4
6. Page Replacement Algorithm — LRU	5
7. Implementation	6
8. Test Results	7
9. Constraints Compliance	8
10. Individual Reflection	9



1. Introduction

This report documents the design and implementation of a Memory Manager for MIT102. The system simulates paged virtual memory with a 1 MB physical memory and a 64 MB backing store, using LRU (Least Recently Used) page replacement an algorithm that is provably immune to Belady's Anomaly.

The implementation is written entirely in Python and comprises three modules:

- `memory_manager.py` the core engine (no system calls used)
- `cli.py` an interactive command-line testing interface
- `tests.py` an automated test suite with 10 annotated test cases

All assignment constraints have been met: single-core assumption, no system calls in the memory manager itself, page size ≤ 4 KB, and a page-replacement algorithm that does not suffer from Belady's Anomaly.

2. System Design & Design Decisions

2.1 Memory Configuration

The following parameters were chosen to satisfy the assignment constraints while maximising efficiency:

Parameter	Value	Justification
Physical Memory	1 MB (1,048,576 B)	Fixed by assignment specification
Backing Store	64 MB (67,108,864 B)	Fixed by assignment specification
Page Size	4 KB (4,096 B)	Maximum allowed; reduces page table overhead and I/O count
Physical Frames	256	$1 \text{ MB} \div 4 \text{ KB} = 256 \text{ frames}$
Backing Frames	16,384	$64 \text{ MB} \div 4 \text{ KB} = 16,384 \text{ swap slots}$
Max Processes	16	Bounds memory consumed by page tables
Logical Limit / Process	4 MB (1,024 virtual pages)	$4 \text{ KB} \times 1,024 = 4 \text{ MB per process}$
Page Replacement	LRU (timestamp-based)	Stack algorithm; immune to Belady's Anomaly

2.2 Why LRU? Belady's Anomaly

Belady's Anomaly is the counter-intuitive observation that some page-replacement algorithms (notably FIFO) can produce more page faults when given more physical frames. LRU avoids this because it belongs to the class of



stack algorithms: the set of pages in memory with k frames is always a subset of the set with $k+1$ frames. This monotonicity property guarantees that adding frames never increases the page-fault rate.

Why Not FIFO? FIFO is simple but suffers from Belady's Anomaly. For example, with the reference string 1-2-3-4-1-2-5-1-2-3-4-5, FIFO produces more faults with 4 frames than with 3. LRU does not exhibit this behaviour.

2.3 Demand Paging

Pages are not loaded into physical memory until they are first accessed. When a process calls `allocate_pages()`, the virtual pages are reserved in the backing store only. The first read or write triggers a page fault, which brings the page into a physical frame. This avoids wasting physical frames on data that is never accessed.

2.4 Dirty Bit & Write-Back Policy

Each page table entry tracks a dirty bit. When a page is evicted, it is written back to the backing store only if dirty (i.e., modified since it was loaded). Clean pages are simply discarded. This minimises unnecessary I/O operations.

3. Memory States

Every virtual page exists in one of four states at any time:

State	Description
INVALID	Page has never been allocated. Accessing it triggers auto-allocation then a page fault.
DISK	Page is allocated but not in physical memory. Resides in the backing store. Access triggers a page fault.
RAM	Page is present in a physical frame. All accesses are served directly.
FREE	Backing store slot has been released (process terminated). Frame returned to the free pool.

3.1 State Transitions

- INVALID $\rightarrow$ DISK: `allocate_pages()` or first auto-allocation on access
- DISK $\rightarrow$ RAM: page fault handler loads page from backing store into a physical frame
- RAM $\rightarrow$ DISK: LRU eviction; dirty pages are written back first
- RAM / DISK $\rightarrow$ FREE: process termination releases all frames

4. Physical Memory Layout

Physical memory is divided into 256 equally-sized frames of 4 KB each. A logical address is translated using:



Formula	Example
$VPN = \text{logical_addr} \div \text{PAGE_SIZE}$	$0x00005A00 \div 4096 = \text{VPN } 5$
$\text{Offset} = \text{logical_addr} \bmod \text{PAGE_SIZE}$	$0x00005A00 \bmod 4096 = 0xA00 \text{ (2560)}$
$\text{Physical addr} = (\text{frame} \times \text{PAGE_SIZE}) + \text{offset}$	frame 12: $(12 \times 4096) + 2560 = 0xCA00$

Logical address spaces are isolated per process — each process's virtual page 0 maps to a different physical frame, providing memory protection. Per-process logical memory is limited to 4 MB (1,024 virtual pages $\times$ 4 KB).

5. Data Structures

5.1 Page Table Entry (PageTableEntry)

One entry exists per virtual page per process.

Field	Type	Purpose
state	str	One of INVALID, DISK, RAM, FREE the current residency of this page
frame_number	int	Physical frame index (if RAM) or backing store slot (if DISK); -1 if INVALID
dirty	bool	Set on any write access; determines whether write-back is needed on eviction
referenced	bool	Set on each access; reserved for future clock/second-chance algorithm upgrade

5.2 Frame Table (FrameInfo)

A 256-entry array, one entry per physical frame. Used by the LRU algorithm and for ownership tracking.

Field	Type	Purpose
pid	int	Owner process ID; -1 if frame is free



vpn	int	Virtual page number currently occupying this frame
last_used	int	Logical clock value of last access; LRU victim has the smallest value

5.3 Backing Store

Implemented as a Python dict mapping backing frame numbers to 4 KB bytearrays. This simulates disk I/O without file system calls. The free list tracks available backing frames (up to 16,384).

5.4 Physical Memory Data

A list of 256 bytearrays (each 4 KB). Index matches frame number. Page content lives here while in RAM; written to backing store on dirty eviction.

6. Page Replacement Algorithm — LRU

6.1 How LRU Works

Every frame maintains a `last_used` timestamp from a monotonically increasing logical clock (incremented on every memory access). When a page fault occurs and no free frame is available, the frame with the smallest `last_used` value is selected as the victim; it is the Least Recently Used page.

6.2 LRU Implementation

```
def lru_victim(self) -> int:
    # Find the frame that was used least recently
    occupied = [(i, f) for i, f in enumerate(self.frames) if f.pid != -1]
    victim_frame, _ = min(occupied, key=lambda x: x[1].last_used)
    return victim_frame

def touch_frame(self, frame_no: int):
    # Update LRU timestamp on every access
    self.frames[frame_no].last_used = self.tick()
```

6.3 Page Fault Sequence

When a read or write targets a page not in RAM, the following steps execute:

- Increment the page fault counter
- Attempt to allocate a free physical frame



- If no free frame: select the LRU victim, write back to backing store if dirty, mark its page table entry as DISK
- Copy the required page from backing store into the selected frame
- Update the page table entry: state = RAM, frame_number = selected frame
- Update the frame table: record pid, vpn, and refresh the LRU timestamp
- Return the frame number to the caller

7. Implementation

7.1 Module Structure

Module / Class	Responsibility
memory_manager.py	Core engine no system calls. Contains all classes below.
PageTableEntry	Stores per-page metadata (state, frame, dirty, referenced)
PhysicalMemory	Manages 256 physical frames, free list, LRU timestamps, and statistics
BackingStore	Simulates 64 MB swap space with a dict; tracks swap-in/swap-out counts
MemoryManager	Top-level: create/terminate processes, read/write bytes, handle page faults
cli.py	Interactive CLI testing interface (system calls allowed here)
tests.py	Automated test suite 10 tests covering all requirements

7.2 Key Operations

- `create_process(name)` allocates a fresh page table with 1,024 INVALID entries. Returns a PID.
- `allocate_pages(pid, n)` reserves n backing-store slots; marks virtual pages as DISK. No physical frame used until first access.
- `read(pid, logical_addr)` translates address via page table, triggers page fault if needed, returns the byte value.
- `write(pid, logical_addr, value)` same as read, but sets the dirty bit and writes the byte.
- `terminate_process(pid)` releases all physical frames and backing-store slots owned by the process.
- `status()` prints a summary: frame usage, fault/eviction/swap counts, per-process RAM/DISK breakdown.

8. Test Results

The following 10 tests were executed against the implementation. All tests passed.



#	Test Name	What Is Verified	Result
1	Process Creation	Three processes created with sequential PIDs; each gets an independent page table (1,024 INVALID entries)	PASS
2	Page Allocation	allocate_pages() marks pages DISK-resident without consuming physical frames (demand paging)	PASS
3	Write → Page Fault	First write to an unloaded page triggers exactly one page fault and transitions the page to RAM	PASS
4	Read → Page Hit	Re-reading the same page produces no additional page fault; correct byte value returned	PASS
5	Cross-page R/W	Writes and reads spanning three different virtual pages all return correct values	PASS
6	Auto-Allocation	Accessing an INVALID page auto-allocates backing store and loads into RAM	PASS
7	LRU Eviction	After filling all 256 frames, accessing a 257th page triggers exactly one LRU eviction	PASS
8	Dirty Write-Back	Evicting a dirty page increments swap_outs; verifies write-back policy is applied	PASS
9	Termination	Terminating a process with 5 loaded pages returns exactly 5 frames to the free pool	PASS



10	Max Process Limit	Attempting to create a 17th process raises MemoryError as per the 16-process limit	PASS
----	-------------------	--	------

Result: 10 / 10 tests passed.

9. Constraints Compliance

Constraint	Required	Implemented
Single-core system (no concurrency)	Yes	Single-threaded Python
No system calls in memory manager	Yes	memory_manager.py uses none
System calls allowed in testing interface	Yes	cli.py uses input(), print()
Page size ≤ 4 KB	Yes	4,096 bytes exactly
Physical memory = 1 MB	Yes	256×4 KB = 1,048,576 B
Backing store = 64 MB	Yes	$16,384 \times 4$ KB = 67,108,864 B
Page replacement immune to Belady's Anomaly	Yes	LRU (stack algorithm)

10. Individual Reflection

What I Learned

Building this memory manager from scratch gave me a much deeper appreciation for the complexity behind a single memory access in a real OS. Concepts like demand paging, dirty bits, and write-back policies are straightforward on paper, but implementing them requires careful attention to state consistency for example, ensuring that when LRU evicts a frame, both the physical frame table and the evicted process's page table entry are updated correctly.

Challenges

The most interesting challenge was choosing the right page replacement algorithm. I considered the Clock (second-chance) algorithm for its efficiency but settled on timestamp-based LRU because it is simpler to reason and unambiguously immune to Belady's Anomaly. The dirty bit write-back policy also required careful handling: an evicted clean page must not trigger a backing-store write, but a dirty one must get this wrong would silently corrupt process data.



Design Trade-offs

The 16-process maximum is a deliberate constraint. Each page table occupies 1,024 entries, and with 16 processes that is 16,384 page table entries in memory. In a real OS, hierarchical or inverted page tables would address scalability, but these were outside the scope of this assignment.

Page size was set to the maximum allowed (4 KB). Smaller pages reduce internal fragmentation but increase the page table size. 4 KB is also the most common choice in real systems (Linux, Windows), making the design realistic.

Future Improvements

A production-quality extension would add: (1) a TLB (Translation Lookaside Buffer) to cache recent VPN→frame translations; (2) working-set tracking for multi-process fairness; and (3) a Clock algorithm for O(1) LRU approximation at scale.